SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5101

COURSE OUTCOME COVERED
CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)
Debugging & Optimisation Task : 10%
QUESTIONS: Total Marks: 50

Annexure A

Code of Conduct:
I **S. Madhuri (192465036)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary
action.

Signature of the student: S. Madhuri

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.
Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.
2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional

handshake failures.
Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.
3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.
Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.
4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.
Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.
5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.
Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION:

Criteria	Weigh tage (%)	Level 4 (Exempla ry)	Level 3 (Proficie nt)	Level 2 (Developi ng)	Level 1 (Beginni ng)
Understa nding of Scenario	20%	Demonst rates clear and complete understa	Shows good understa nding with	Partial understan ding; misses key	Misinter prets or fails to understa nd the

		nding of the scenario and context	minor gaps	aspects	scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: Debugging Hash Function Implementation (SHA Family)

Problem identification and root causes

- Inconsistent output for identical input usually means the program is not hashing the same bytes, the message buffer is being changed, or custom padding/block processing is incorrect. A correct deterministic hash produces the same digest for the same byte sequence.
- Padding errors include appending the wrong bit length, using bytes instead of bits, placing the length field in the wrong byte order, or adding an incorrect number of zero bits.
- Block-processing errors include skipping or duplicating a block, mishandling a final partial block, or resetting the hash state between blocks.
- Bitwise errors include using an incorrect rotation instead of a shift, failing to mask values to the required word width in a hand-written implementation, and applying operations in the wrong order.
- Endianness errors occur when 32-bit or 64-bit words are parsed or serialized in the wrong byte order. SHA-256 specifies big-endian word interpretation.
- Reading a file as text can alter bytes through character decoding or newline conversion. Files must be processed as binary data.

1.2 Corrected implementation

Use Python's maintained hashlib implementation instead of writing SHA padding and compression manually. Streaming in chunks avoids loading a large file into memory.

```
import hashlib
def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()
def sha256_file(path: str, chunk_size: int = 1024 * 1024) -> str:
    digest = hashlib.sha256()
    with open(path, "rb") as file:
        while True:
            chunk = file.read(chunk_size)
            if not chunk:
                break
            digest.update(chunk)
    return digest.hexdigest()
print("Text digest:", sha256_bytes(b"hello"))
print("File digest:", sha256_file("large_file.bin"))
```

```python
def sha256_file(path: str, chunk_size: int = 1024 * 1024) -> str:
    if not os.path.exists(path):
        return f"Error: File '{path}' not found."
    digest = hashlib.sha256()
    with open(path, "rb") as file:
        while True:
            chunk = file.read(chunk_size)
            if not chunk:
                break
            digest.update(chunk)
    return digest.hexdigest()
test_filename = "large_file.bin"
with open(test_filename, "wb") as f:
    f.write(b"This is a simulated large file for testing SHA-256 hashing.")
print("Text digest:", sha256_bytes(b"hello"))
print("File digest:", sha256_file(test_filename))
if os.path.exists(test_filename):
    os.remove(test_filename)
```

```
Text digest: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
File digest: 62db39210d491064ca8f40fb1498931603ccdb6b538c2864a5ec94567cb199a5
```

1.3 Debugging and testing procedure

- Test known inputs such as the empty byte string and b'abc'; compare results with a trusted SHA-256 test vector or an independent trusted tool.
- Hash the same file twice and confirm that both digests match.
- Change one byte and confirm that the digest changes substantially.
- Test file sizes below one chunk, exactly one chunk, and slightly above one chunk to check block-boundary handling.
- Compare a small file's streaming result with hashlib.sha256(open(path, 'rb').read()).hexdigest(). The results must match.

1.4 Optimization for files of 50 MB or more

- Use hashlib, which calls optimized native code, instead of Python-level bit-by-bit processing.
- Read fixed-size chunks (for example, 1 MiB) to keep memory use approximately constant regardless of file size.
- Avoid converting binary data to strings, repeated concatenation, and unnecessary copies.
- Benchmark using time.perf_counter() and report actual results for the test computer; do not claim a speed-up without measuring it.
- For very large files, storage speed may be the main bottleneck. A larger chunk may help, but the best size should be measured.

1.5 Security analysis: hash versus checksum

A basic checksum is useful for accidental transmission errors, but it is not designed to resist deliberate modification. An attacker may be able to change data and recompute a simple checksum. SHA-256 is a cryptographic hash designed to make it computationally infeasible to find practical collisions or recover an input from its digest. A digest can detect a changed file when compared with a trusted digest, but it does not authenticate who created the digest. For authenticity, use a digital signature or a keyed MAC such as HMAC.

1.6 Expected result

The same byte sequence consistently produces the same SHA-256 digest. Streaming permits large-file hashing with bounded memory use, and the result can be compared with a trusted reference to verify integrity.

Question 2: Optimization of SSL/TLS Simulation

2.1 Problem identification and root causes

- Certificate failures can result from an untrusted certificate chain, hostname mismatch, expired/not-yet-valid certificates, missing intermediate certificates, or an incorrect system clock.
- Handshake failures can arise from unsupported protocol versions or cipher suites, invalid key shares, malformed messages, or incorrect client/server state transitions.
- High latency may be caused by repeated full handshakes, extra network round trips, repeated DNS/TCP connection setup, or expensive certificate validation on every new connection.
- Disabling certificate validation or accepting every certificate may appear to fix failures, but it removes essential protection against man-in-the-middle attacks and is not an acceptable optimization.

2.2 Safe Python TLS client example

The standard ssl module verifies certificates and hostnames when the default verified context is used. This example demonstrates a correctly configured TLS connection; the server must be one you are authorized to contact.

```
import socket
import ssl
import time
def tls_request(host: str, port: int = 443) -> float:
    context = ssl.create_default_context()
    start = time.perf_counter()
    with socket.create_connection((host, port), timeout=10) as raw:
        with context.wrap_socket(raw, server_hostname=host) as tls:
            tls.sendall(
                f"HEAD / HTTP/1.1\\r\\nHost: {host}\\r\\n"
                "Connection: close\\r\\n\\r\\n".encode("ascii")
            )
            response = tls.recv(4096)
            print("TLS version:", tls.version())
            print("Cipher:", tls.cipher())
```

```
        print("Response received:", bool(response))
    return time.perf_counter() - start

# Example: use a permitted test server
# print("Elapsed seconds:", tls_request("example.com"))
```

```python
import socket
import ssl
import time
def tls_request(host: str, port: int = 443) -> float:
    context = ssl.create_default_context()
    start = time.perf_counter()
    with socket.create_connection((host, port), timeout=10) as raw:
        with context.wrap_socket(raw, server_hostname=host) as tls:
            tls.sendall(
                f"HEAD / HTTP/1.1\r\nHost: {host}\r\n"
                "Connection: close\r\n\r\n".encode("ascii")
            )
            response = tls.recv(4096)
            print("TLS version:", tls.version())
            print("Cipher:", tls.cipher())
            print("Response received:", bool(response))
    return time.perf_counter() - start
print("Elapsed seconds:", tls_request("example.com"))
```

```
TLS version: TLSv1.3
Cipher: ('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256)
Response received: True
Elapsed seconds: 0.07840842799998882
```

2.3 Debugging and optimization plan

- Log the negotiated TLS version, cipher suite, exception type, and certificate verification error without logging private keys or sensitive session material.
- Keep certificate and hostname verification enabled. Confirm that the client connects using the hostname present in the certificate.
- Prefer TLS 1.3 where supported, and maintain a modern, server-supported cipher configuration.
- Reuse an established connection for multiple requests when the application protocol supports persistent connections; this avoids repeating a handshake.
- Use TLS session resumption only through the TLS library's supported mechanisms. Do not invent or manually reuse session keys.
- Set sensible connection timeouts and distinguish DNS, TCP-connect, TLS-handshake, and application-response time in measurements.

2.4 Session resumption and trade-offs

Session resumption can reduce handshake work and latency when a client reconnects to the same service. TLS 1.3 resumption commonly uses tickets and may support a 0-RTT mode. Early data can be replayed, so it must not be used for non-idempotent actions such as transferring money or making a one-time state change. Tickets should be protected, rotated, and configured with reasonable lifetimes. Resumption does not justify disabling certificate checks or weakening protocol settings.

2.5 Evaluation

Measure full-handshake latency and resumed-handshake latency across repeated trials, and report the median and a high percentile such as p95. Also record failure rate and negotiated TLS version. An optimization is acceptable only if it lowers latency while preserving certificate validation, modern cryptography, and appropriate replay protections.

2.6 Expected result

Correct validation removes avoidable certificate errors, connection reuse reduces repeated setup, and supported session resumption can reduce reconnection latency without manually exposing or reusing raw session keys.

Question 3: Debugging and Optimizing Digital Signature Algorithm (DSA)

3.1 Problem identification and root causes

- DSA parameters must satisfy the algorithm's mathematical requirements; incompatible or incorrectly generated parameters lead to invalid signatures.
- The private key x must be in the permitted range and the public key must be computed consistently from the parameters.
- The per-message secret nonce k must be unpredictable, unique for every signature under a key, and in the correct range. Reusing k can reveal the private key.
- Common implementation errors include using the wrong message digest, mixing integer and byte encodings, using an incorrect modular inverse, or verifying with parameters different from those used to sign.
- A signature is invalid if either signature component is out of range or if the verification equation is evaluated incorrectly.

3.2 Safer modern implementation

For new systems, prefer a modern signature scheme supported by a maintained library, such as Ed25519. DSA is legacy and should only be implemented where a course or compatibility requirement explicitly calls for it. The following code demonstrates signing and verification with the Python cryptography package.

```
# Install once in the environment: pip install cryptography
from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey
)
```

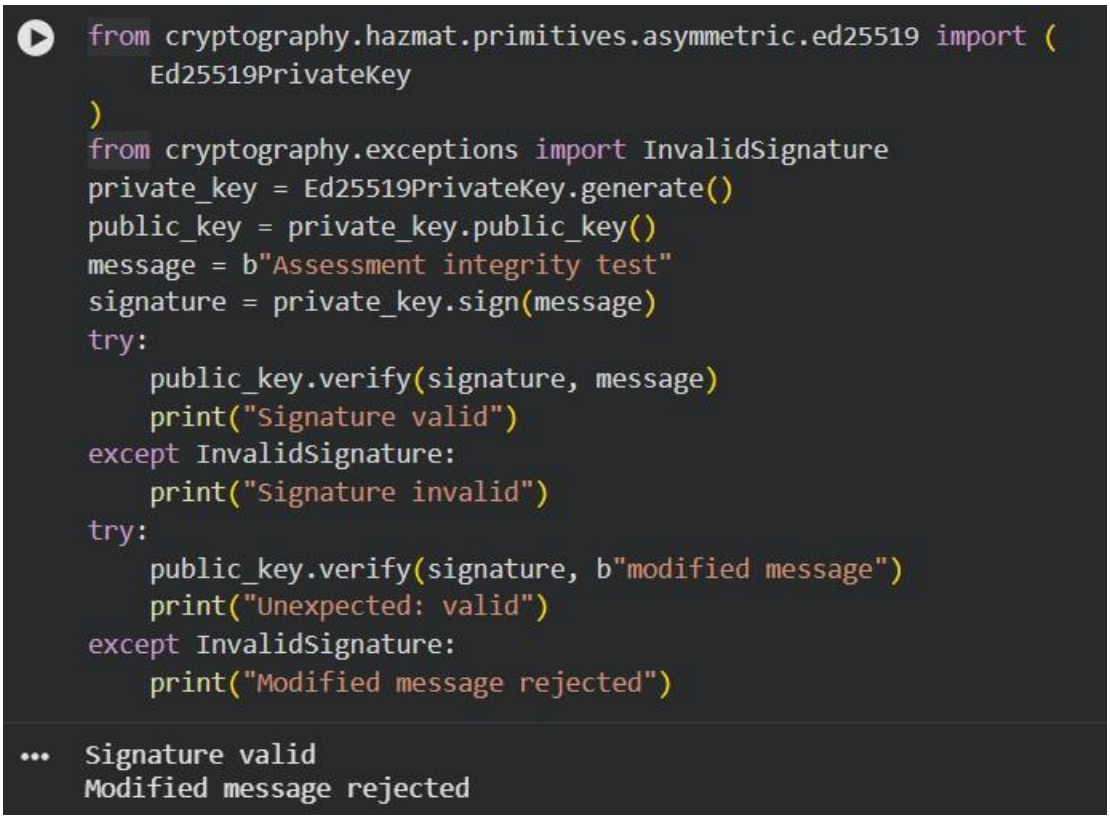
```
from cryptography.exceptions import InvalidSignature

private_key = Ed25519PrivateKey.generate()
public_key = private_key.public_key()
message = b"Assessment integrity test"

signature = private_key.sign(message)

try:
    public_key.verify(signature, message)
    print("Signature valid")
except InvalidSignature:
    print("Signature invalid")

# A changed message must fail verification.
try:
    public_key.verify(signature, b"modified message")
    print("Unexpected: valid")
except InvalidSignature:
    print("Modified message rejected")
```



3.3 If DSA is specifically required

- Use a vetted cryptographic library's DSA key-generation, signing, and verification APIs rather than manually implementing nonce generation.
- Use approved parameter sizes and a secure digest according to the applicable standard and library guidance.

- Never generate k with a basic pseudorandom function, timestamp, or predictable seed. Use the library's secure nonce handling.
- Validate signature component ranges and ensure that the exact same message bytes and parameters are used at verification.
- Write negative tests: alter the message, signature, or public key and confirm verification fails.

3.4 Performance optimization

Modular exponentiation should use the language or library's optimized modular-power operation, such as pow(base, exponent, modulus), rather than calculating a huge power and applying the modulus afterward. Modular inverses and key operations should use library routines. Avoid optimizing by reducing key sizes or weakening nonce generation. For bulk data, sign a cryptographic digest through a supported high-level API rather than manually hashing inconsistently.

3.5 Randomness and security analysis

Signature security depends on nonce secrecy and uniqueness. If a DSA nonce is reused or becomes predictable, an attacker may recover the private signing key and forge signatures. A strong random generator and vetted implementation are therefore critical. Ed25519 avoids application-managed per-message nonce generation and is generally a better choice for new applications.

3.6 Expected result

Valid messages verify successfully, while modified messages or signatures are rejected. Using a maintained implementation avoids common arithmetic and randomness mistakes and reduces the risk of private-key compromise.

Question 4: Debugging Key Management System

4.1 Problem identification and root causes

- Storing keys as plaintext in source code, configuration files, logs, or a database exposes them to anyone who obtains those files or access.
- Using a fixed key, a weak password-derived key, or an insecure random generator can make encrypted data recoverable.
- Improper access control may allow unauthorized users or services to retrieve keys.
- Missing rotation, revocation, backup, or destruction procedures can leave old or compromised keys active.
- Encrypting a key with another key stored beside it in plaintext provides little protection. Key-encryption keys must be separately protected.

4.2 Safer key storage example

For a small educational demonstration, Fernet provides authenticated encryption for a stored secret. In production, prefer an operating-system keystore, cloud KMS, or hardware security module, with access control and auditing. Never commit the encryption key to the same source repository as the encrypted file.

```
# Install once in the environment: pip install cryptography
import os
from pathlib import Path
from cryptography.fernet import Fernet

KEY_PATH = Path("key_encryption.key")
DATA_PATH = Path("protected_secret.bin")

def create_key_if_missing():
    if not KEY_PATH.exists():
        # Demo only: protect this file with strict OS permissions.
        KEY_PATH.write_bytes(Fernet.generate_key())
        try:
            os.chmod(KEY_PATH, 0o600)
        except OSError:
            pass

def store_secret(secret: bytes):
    create_key_if_missing()
    key = KEY_PATH.read_bytes()
    encrypted = Fernet(key).encrypt(secret)
    DATA_PATH.write_bytes(encrypted)

def retrieve_secret() -> bytes:
    key = KEY_PATH.read_bytes()
    encrypted = DATA_PATH.read_bytes()
    return Fernet(key).decrypt(encrypted)

if __name__ == "__main__":
    store_secret(b"demo API credential")
    print("Recovered:", retrieve_secret().decode())
```

```python
import os
from pathlib import Path
from cryptography.fernet import Fernet
KEY_PATH = Path("key_encryption.key")
DATA_PATH = Path("protected_secret.bin")
def create_key_if_missing():
    if not KEY_PATH.exists():
        KEY_PATH.write_bytes(Fernet.generate_key())
        try:
            os.chmod(KEY_PATH, 0o600)
        except OSError:
            pass
def store_secret(secret: bytes):
    create_key_if_missing()
    key = KEY_PATH.read_bytes()
    encrypted = Fernet(key).encrypt(secret)
    DATA_PATH.write_bytes(encrypted)
def retrieve_secret() -> bytes:
    key = KEY_PATH.read_bytes()
    encrypted = DATA_PATH.read_bytes()
    return Fernet(key).decrypt(encrypted)
if __name__ == "__main__":
    store_secret(b"demo API credential")
    print("Recovered:", retrieve_secret().decode())
```

```
Recovered: demo API credential
```

4.3 Debugging and optimization

- Separate key creation, storage, retrieval, rotation, revocation, and destruction into auditable functions.
- Restrict key-file or KMS access to the minimum required users and services; use least privilege and multi-factor authentication for administrative access.
- Do not print keys, decrypted secrets, access tokens, or encryption material in logs.
- Use envelope encryption for larger systems: encrypt data with a data-encryption key and protect that key with a separately managed key-encryption key.
- Plan rotation and re-encryption before deployment. Keep version metadata so data can be decrypted with the correct authorized key version.
- Test wrong-key, tampered-ciphertext, missing-key, revoked-key, and unauthorized-access cases.

4.4 Key lifecycle

Generation: create keys with a cryptographically secure generator. Distribution: transfer them through authenticated, protected channels or a KMS. Storage: encrypt and restrict access. Use: expose keys only to the process that needs them and for the shortest practical time. Rotation: create a new version, migrate dependents, and retire the old version. Revocation: immediately disable compromised keys and investigate affected data. Destruction: securely remove retired key material according to policy, including managed backups where applicable.

4.5 Analysis

Cryptographic algorithms cannot protect data if their keys are exposed. A stolen key can let an attacker decrypt stored information, impersonate a service, or forge authentication data. Centralized key management, least privilege, auditing, rotation, and tested recovery procedures reduce this risk. The code above is a learning example, not a replacement for a managed production KMS.

4.6 Expected result

The demonstration stores an authenticated encrypted secret and recovers it only when the correct key is available. Production deployment should use a managed key service and enforce a complete lifecycle policy.

Question 5: Optimization of Secure File Transfer (SFTP-like System)

5.1 Problem identification and root causes

- Data corruption may be caused by incorrect chunk offsets, dropped or duplicated packets, incorrect reassembly, or writing chunks in the wrong order.
- Encryption/decryption synchronization can fail when the sender and receiver disagree about framing, nonce/IV, authentication tag, or message boundaries.
- Using one giant in-memory buffer increases memory consumption for files of 100 MB or more.
- Repeatedly creating cryptographic contexts, using tiny chunks, or performing unnecessary copies can reduce throughput.
- Encryption without authentication may not detect malicious modification. Integrity checks must be cryptographic and verified before accepting plaintext.

5.2 Recommended secure solution

For real transfers, use maintained SFTP (SSH File Transfer Protocol) software or a properly configured TLS-based protocol rather than designing a new transport protocol. The following local educational example uses authenticated encryption per chunk. It demonstrates bounded-memory processing and rejects tampering; it is not a network protocol and should not be described as a complete SFTP replacement.

```
# Install once in the environment: pip install cryptography
import os
import struct
from pathlib import Path
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

CHUNK = 1024 * 1024  # 1 MiB
MAGIC = b"SAF1"

def encrypt_file(source, destination, key):
    aes = AESGCM(key)
    with open(source, "rb") as src, open(destination, "wb") as dst:
```

```
        dst.write(MAGIC)
        index = 0
        while True:
            block = src.read(CHUNK)
            if not block:
                break
            nonce = os.urandom(12)
            aad = struct.pack(">Q", index)
            ciphertext = aes.encrypt(nonce, block, aad)
            dst.write(struct.pack(">I", len(ciphertext)))
            dst.write(nonce)
            dst.write(ciphertext)
            index += 1

def decrypt_file(source, destination, key):
    aes = AESGCM(key)
    with open(source, "rb") as src, open(destination, "wb") as dst:
        if src.read(4) != MAGIC:
            raise ValueError("Invalid file format")
        index = 0
        while True:
            length_bytes = src.read(4)
            if not length_bytes:
                break
            if len(length_bytes) != 4:
                raise ValueError("Truncated length")
            length = struct.unpack(">I", length_bytes)[0]
            nonce = src.read(12)
            ciphertext = src.read(length)
            if len(nonce) != 12 or len(ciphertext) != length:
                raise ValueError("Truncated chunk")
            aad = struct.pack(">Q", index)
            plaintext = aes.decrypt(nonce, ciphertext, aad)
            dst.write(plaintext)
            index += 1

# Example usage
# key = AESGCM.generate_key(bit_length=256)
# encrypt_file("input.bin", "input.sec", key)
# decrypt_file("input.sec", "recovered.bin", key)
```

```python
import os
import struct
from pathlib import Path
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
CHUNK = 1024 * 1024  # 1 MiB
MAGIC = b"SAF1"
def encrypt_file(source, destination, key):
    aes = AESGCM(key)
    with open(source, "rb") as src, open(destination, "wb") as dst:
        dst.write(MAGIC)
        index = 0
        while True:
            block = src.read(CHUNK)
            if not block:
                break
            nonce = os.urandom(12)
            aad = struct.pack(">Q", index)
            ciphertext = aes.encrypt(nonce, block, aad)
            dst.write(struct.pack(">I", len(ciphertext)))
            dst.write(nonce)
            dst.write(ciphertext)
            index += 1
def decrypt_file(source, destination, key):
    aes = AESGCM(key)
    with open(source, "rb") as src, open(destination, "wb") as dst:
        if src.read(4) != MAGIC:
            raise ValueError("Invalid file format")
        index = 0
        while True:
            length_bytes = src.read(4)
            if not length_bytes:
                break
            if len(length_bytes) != 4:
                raise ValueError("Truncated length")
            length = struct.unpack(">I", length_bytes)[0]
            nonce = src.read(12)
            ciphertext = src.read(length)
            if len(nonce) != 12 or len(ciphertext) != length:
                raise ValueError("Truncated chunk")
            aad = struct.pack(">Q", index)
            plaintext = aes.decrypt(nonce, ciphertext, aad)
            dst.write(plaintext)
            index += 1
input_file = "input.bin"
with open(input_file, "wb") as f:
    f.write(b"This is some test data to encrypt and decrypt using authenticated AES-GCM encryption.")
key = AESGCM.generate_key(bit_length=256)
encrypt_file(input_file, "input.sec", key)
decrypt_file("input.sec", "recovered.bin", key)
with open("recovered.bin", "rb") as f:
    decrypted_content = f.read()
print("Decryption successful! Decrypted content:")
print(decrypted_content.decode("utf-8"))
for filename in ["input.bin", "input.sec", "recovered.bin"]:
    if os.path.exists(filename):
        os.remove(filename)
```

```
Decryption successful! Decrypted content:
This is some test data to encrypt and decrypt using authenticated AES-GCM encryption.
```

5.3 Important correctness and security conditions

- Keep the 256-bit AES key secret and transfer it through a separate authenticated key-management mechanism.
- Generate a unique 12-byte nonce for every encryption under the same AES-GCM key. Never reuse a nonce with that key.
- The receiver authenticates each chunk before writing its plaintext. Authentication failure raises an exception.
- This simple format does not authenticate the entire file's ordering, truncation at a clean chunk boundary, or metadata. A production format must authenticate a final manifest containing chunk count, total length, file identity, and ordering, or use a mature protocol.
- Use temporary output and rename it only after the complete transfer and final verification succeed, so a failed transfer does not leave a seemingly complete file.
- Use binary mode, exact-length reads, explicit framing, bounded buffers, and clear error handling.

5.4 Optimization for files of 100 MB or more

- Stream in chunks instead of reading the complete file into memory; memory use remains proportional to the chosen chunk size.
- Benchmark chunk sizes such as 64 KiB, 256 KiB, and 1 MiB on the target hardware and network.
- Reuse the AESGCM object while using a fresh nonce for every chunk. Do not reuse nonces to gain speed.
- Avoid repeated string conversions and extra copies. For networked transfers, pipeline or buffer I/O carefully while preserving sequence numbers and authentication.
- Measure total time, throughput (MB/s), peak memory, and corruption/tampering detection. Test a 100 MB file and compare hashes of the original and recovered files.

5.5 Comparison with insecure transfer methods

Feature	Plain FTP / unprotected copy	SFTP or authenticated encrypted transfer
Confidentiality	Credentials/data may be exposed in transit	Encryption protects data in transit
Integrity	No reliable cryptographic tamper protection by default	Authenticated encryption/protocol integrity detects tampering
Authentication	Plain FTP does not inherently provide strong server identity protection	SSH host-key verification or TLS certificate validation authenticates the endpoint
Performance	May have low cryptographic overhead but offers weaker protection	Some encryption overhead; connection reuse and streaming improve practical throughput
Recommendation	Avoid for sensitive information on untrusted networks	Use maintained, securely configured software and verified host identities

5.6 Testing and expected result

- Encrypt and decrypt a test file; compare SHA-256 digests of the original and recovered files.
- Test empty, small, exact-chunk, multi-chunk, and 100 MB files.
- Flip a byte in a ciphertext chunk and confirm authenticated decryption fails.
- Truncate the encrypted file and confirm the program reports an error; add a final authenticated manifest in a production-quality implementation to detect all forms of truncation.
- Report measured throughput and memory use from the actual environment rather than inventing performance numbers.

Expected result: streaming reduces memory usage for large files, authenticated encryption detects chunk tampering, and a mature SFTP implementation provides a safer, more complete protocol for real transfers.

Overall Conclusion

The five tasks show that cryptographic reliability depends on correct implementation, safe defaults, secure randomness, disciplined key management, and systematic testing. Hashing should use a trusted SHA implementation; TLS should retain certificate validation while reducing unnecessary handshakes; signatures should use vetted algorithms and secure nonce handling; keys should be protected throughout their lifecycle; and file transfers should use authenticated encryption or a maintained protocol such as SFTP. Performance should be demonstrated with repeatable measurements, while security checks must never be removed merely to make a program faster.

Suggested Rubric Evidence Checklist

- Problem identification: list the symptom, root cause, and security impact for each question.
- Debugging: include positive and negative test cases and show the expected outcome.
- Optimization: record baseline and optimized runtime, throughput, memory use, and test environment.
- Analysis: explain why the fix works and identify any remaining limitations.
- Code quality: use functions, meaningful names, exception handling, clear comments, and maintained cryptographic libraries.